

# *Social Computing*

## *17. Graph Visualization with Pyvis*

*Hafsa AKEBLI*

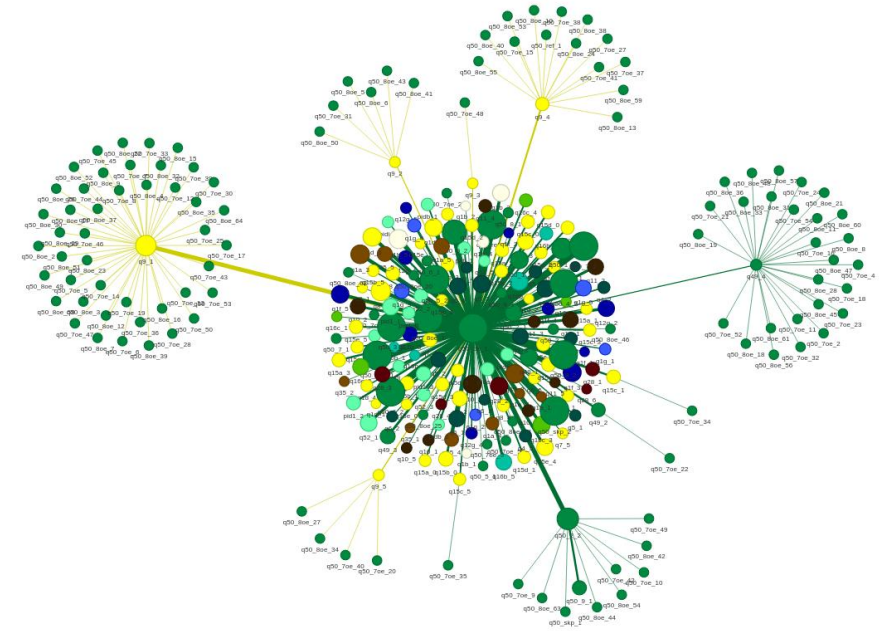
*Date: 28/11/2025*



# Summary

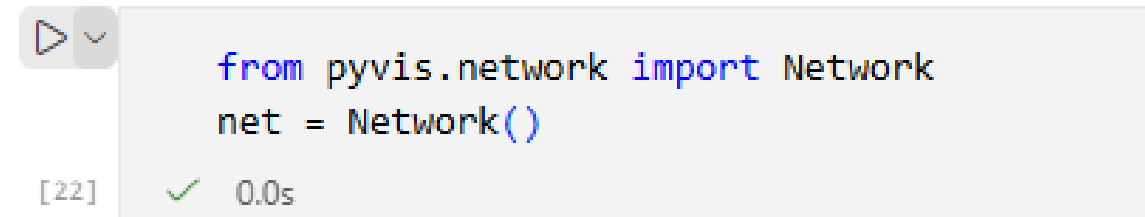
- What is **Pyvis**
- Getting Started with **Pyvis**
- Adding nodes/list of nodes to the Network, with/without properties
- Adding Edges to the Network with/without properties
- Integrating with **NetworkX**
- **Game of Graphs** visualization
- Graph Measures

- **PyVis** is a Python library for creating and visualizing interactive network graphs
- It allows you to explore graphs **directly in the browser**
- You can: Drag nodes, Zoom and move around the graph, Click and inspect connections
- Suitable for **visualizing large graphs** in an intuitive way
- Useful for: Social networks, Relationship data, Understanding graph structures



# Getting started with Pyvis

- **Installing pyvis with** : `pip install pyvis`
- **Importing the library**
- A PyVis graph must be created as a **Network object**
- The **Network object** will store: Nodes, Edges and Visual Style
- Everything we do starts from: `net = Network()`



```
from pyvis.network import Network
net = Network()
```

[22] ✓ 0.0s

# Adding Nodes to the Network

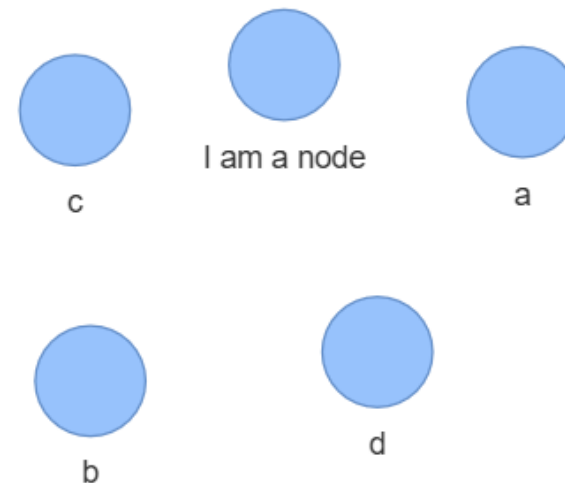
- Use `add_node()` to add **one node**
- First parameter = **node ID**
  - can be a number/string
  - must be unique
- **label** is the text shown on the graph
- If no label is given → the ID is used as label
- Use `add_nodes()` to add many nodes at once;  
Accepts a list (or any iterable) of nodes **IDs**

```
from pyvis.network import Network
net = Network()
```

[1] ✓ 0.1s

```
#adding nodes to the Network
net.add_node(1, label="I am a node")
#adding nodes from a list
nodes = ["a", "b", "c", "d"]
net.add_nodes(nodes)
net.write_html("example_adding_nodes.html")
```

[9] ✓ 0.0s



# Adding list of nodes with properties

- You can add visual and positional properties to nodes using **add\_nodes()**
- Each property list must have the **same length** as the list of node IDs
- Common node properties:
  - label** : text shown on the graph (*always visible*)
  - color** : node color
  - value** : node size
  - title** : node description or name (*appears only when you hover the mouse over the node in the html view*)
  - x, y** : coordinates (optional manual positioning)
- These properties help customize appearance and add information to the graph

```
from pyvis.network import Network

# Aggiungere nodi con proprietà (esempio con monumenti italiani)
h = Network(height="400px", width="600px")

h.add_nodes(
    ["Colosseo", "Duomo", "Torre di Pisa"],
    value=[10, 100, 400],
    title=[
        "Roma ☐ Colosseo",
        "Milano ☐ Duomo",
        "Pisa ☐ Torre pendente"
    ],
    x=[20, 70, 10],
    y=[90, 30, 50],
    label=["Colosseo", "Duomo", "Torre di Pisa"],
    color=[ "#FFD700", "#DC143C", "#00BFFF" ]
)

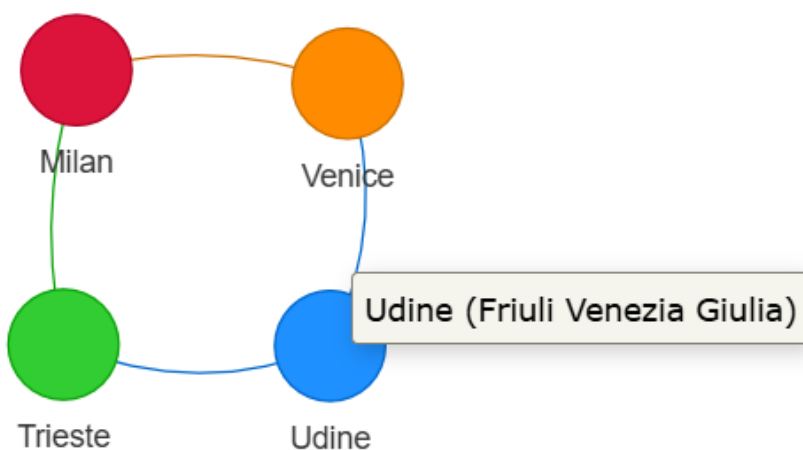
h.write_html("node_properties_example.html")
```

✓ 0.0s



# Adding Edges to the Network

- Edges connect nodes using their **IDs**
- Use **add\_edge(source, target)** to create a connection
- You can add **weights** or visual properties (e.g. color, width) to the edge



```

#adding edges Example
from pyvis.network import Network

# Create a small network of Italian cities
net = Network(height="400px", width="600px")

# Add nodes (cities)
net.add_nodes(
    ["Udine", "Trieste", "Venice", "Milan"],
    color=["#1E90FF", "#32CD32", "#FF8C00", "#DC143C"],
    title=[
        "Udine (Friuli Venezia Giulia)",
        "Trieste (Friuli Venezia Giulia)",
        "Venice (Veneto)",
        "Milan (Lombardy)"
    ]
)

# Add edges with real distances (in km)
net.add_edge("Udine", "Trieste", weight=79, title="79 km")
net.add_edge("Udine", "Venice", weight=132, title="132 km")
net.add_edge("Venice", "Milan", weight=278, title="278 km")
net.add_edge("Trieste", "Milan", weight=423, title="423 km")

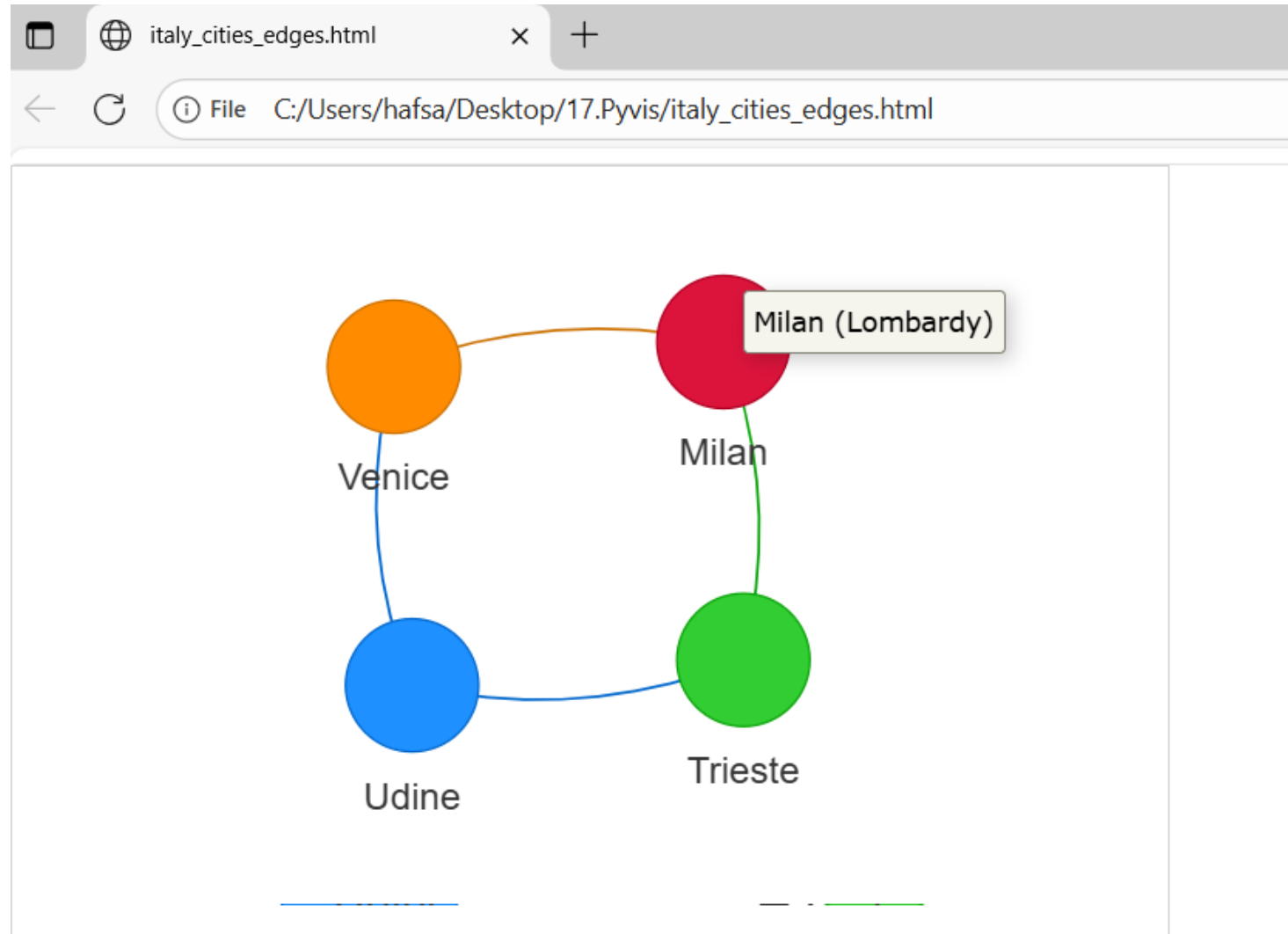
# Save as HTML file
net.write_html("italy_cities_edges.html")

```

✓ 0.0s

# Adding Edges to the Network

HTML FILE



# Integrating with NetworkX

- **PyVis** can directly visualize **NetworkX** graphs
- Use **.from\_nx()** to import any existing NetworkX graph
- Node and edge attributes in **NetworkX** (like title, group, size, color) are automatically translated and reused by **PyVis**.
- This makes it easy to move from **analysis (NetworkX)** to **interactive and dynamic visualization (PyVis)**

```
Click to add a breakpoint import Network
import networkx as nx

# Create a NetworkX graph of research collaborations
G = nx.Graph()

# Add edges with the number of collaborations as weight
G.add_edge("Alice", "Bob", weight=3)      # 3 papers together
G.add_edge("Alice", "Carla", weight=1)
G.add_edge("Bob", "David", weight=2)
G.add_edge("Carla", "David", weight=4)
G.add_edge("Alice", "David", weight=1)

# Add roles and info as node attributes
G.nodes["Alice"]["title"] = "Alice [ PhD student"
G.nodes["Bob"]["title"] = "Bob [ Post-doc"
G.nodes["Carla"]["title"] = "Carla [ Associate Professor"
G.nodes["David"]["title"] = "David [ Professor"

# Optional: color nodes by role
G.nodes["Alice"]["color"] = "#00BFFF"
G.nodes["Bob"]["color"] = "#32CD32"
G.nodes["Carla"]["color"] = "#FFD700"
G.nodes["David"]["color"] = "#FF4500"

# Convert and visualize with PyVis
net = Network(height="400px", width="600px")
net.from_nx(G)

net.write_html("networkx_collaboration_example.html")

✓ 0.0s
```

# Integrating with NetworkX

```
from pyvis.network import Network
import networkx as nx

# Create a NetworkX graph of research collaborations
G = nx.Graph()

# Add edges with the number of collaborations as weight
G.add_edge("Alice", "Bob", weight=3)      # 3 papers together
G.add_edge("Alice", "Carla", weight=1)
G.add_edge("Bob", "David", weight=2)
G.add_edge("Carla", "David", weight=4)
G.add_edge("Alice", "David", weight=1)

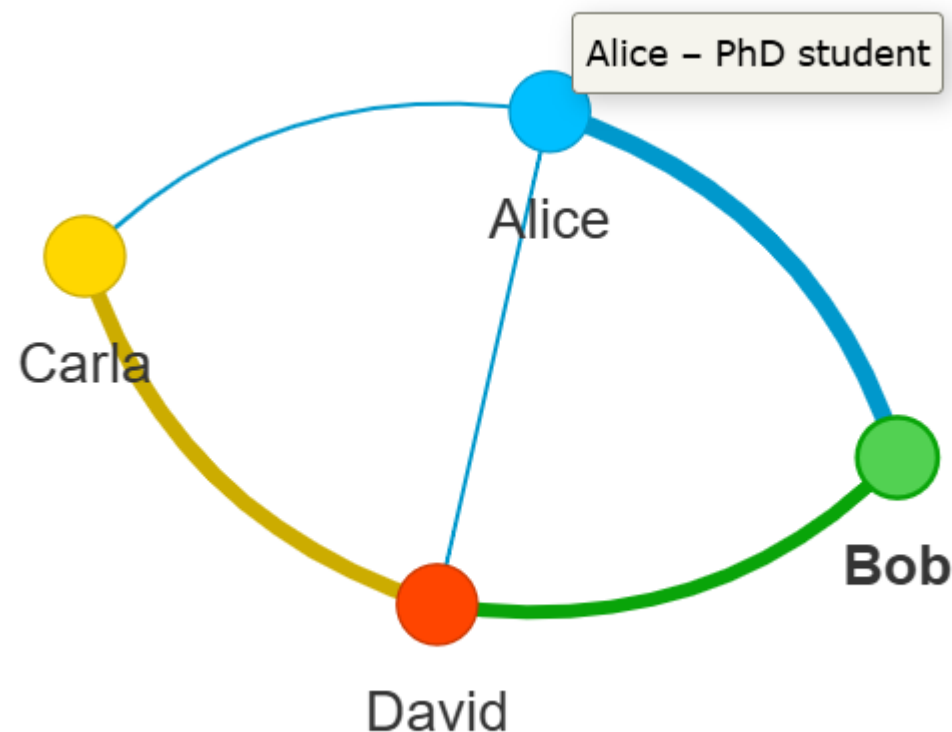
# Add roles and info as node attributes
G.nodes["Alice"]["title"] = "Alice ☐ PhD student"
G.nodes["Bob"]["title"] = "Bob ☐ Post-doc"
G.nodes["Carla"]["title"] = "Carla ☐ Associate Professor"
G.nodes["David"]["title"] = "David ☐ Professor"

# Optional: color nodes by role
G.nodes["Alice"]["color"] = "#00BFFF"
G.nodes["Bob"]["color"] = "#32CD32"
G.nodes["Carla"]["color"] = "#FFD700"
G.nodes["David"]["color"] = "#FF4500"

# Convert and visualize with PyVis
net = Network(height="400px", width="600px")
net.from_nx(G)

net.write_html("networkx_collaboration_example.html")
```

✓ 0.0s



**Nodes** = researchers (with their roles)

**Edges** = co-authored papers

**Weight** = number of collaborations

the **weight** parameter **controls the edge thickness**

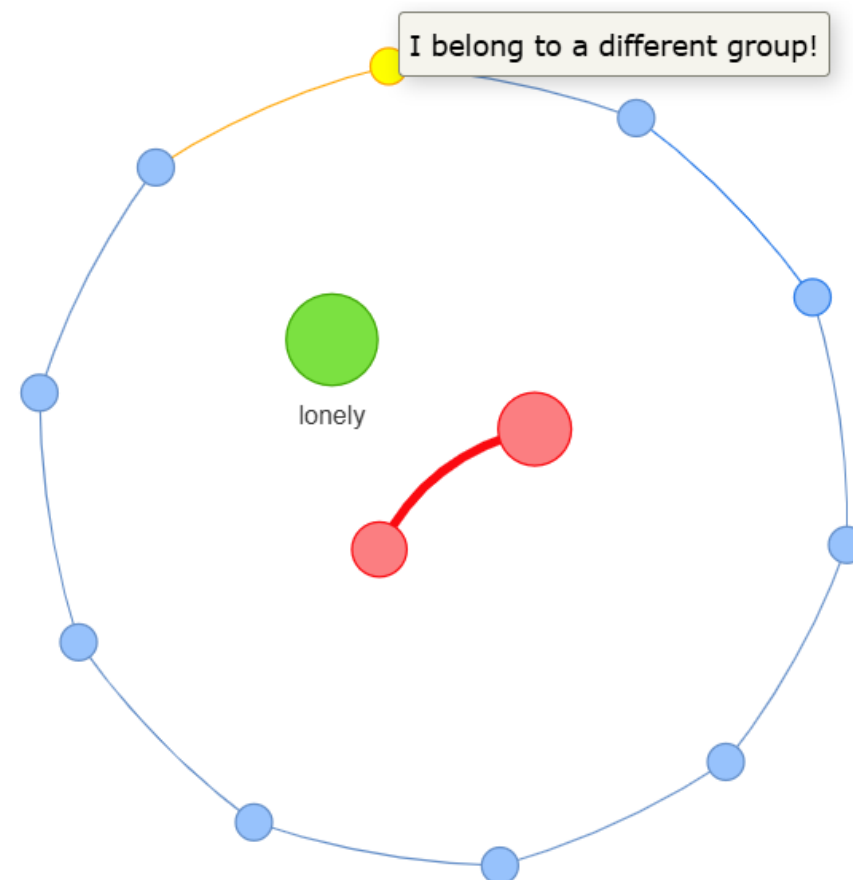
# Integrating with NetworkX

```

• ✓ from pyvis.network import Network
    import networkx as nx
    nx_graph = nx.cycle_graph(10)
    nx_graph.nodes[1]['title'] = 'Number 1'
    nx_graph.nodes[1]['group'] = 1
    nx_graph.nodes[3]['title'] = 'I belong to a different group!'
    nx_graph.nodes[3]['group'] = 10
    nx_graph.add_node(20, size=20, title='couple', group=2)
    nx_graph.add_node(21, size=15, title='couple', group=2)
    nx_graph.add_edge(20, 21, weight=5)
    nx_graph.add_node(25, size=25, label='lonely', title='lonely node', group=3)
    nt_1 = Network('500px', '500px')
    # populates the nodes and edges data structures
    nt_1.from_nx(nx_graph)
    nt_1.write_html("networkx_cycle_graph.html")

```

✓ 0.0s

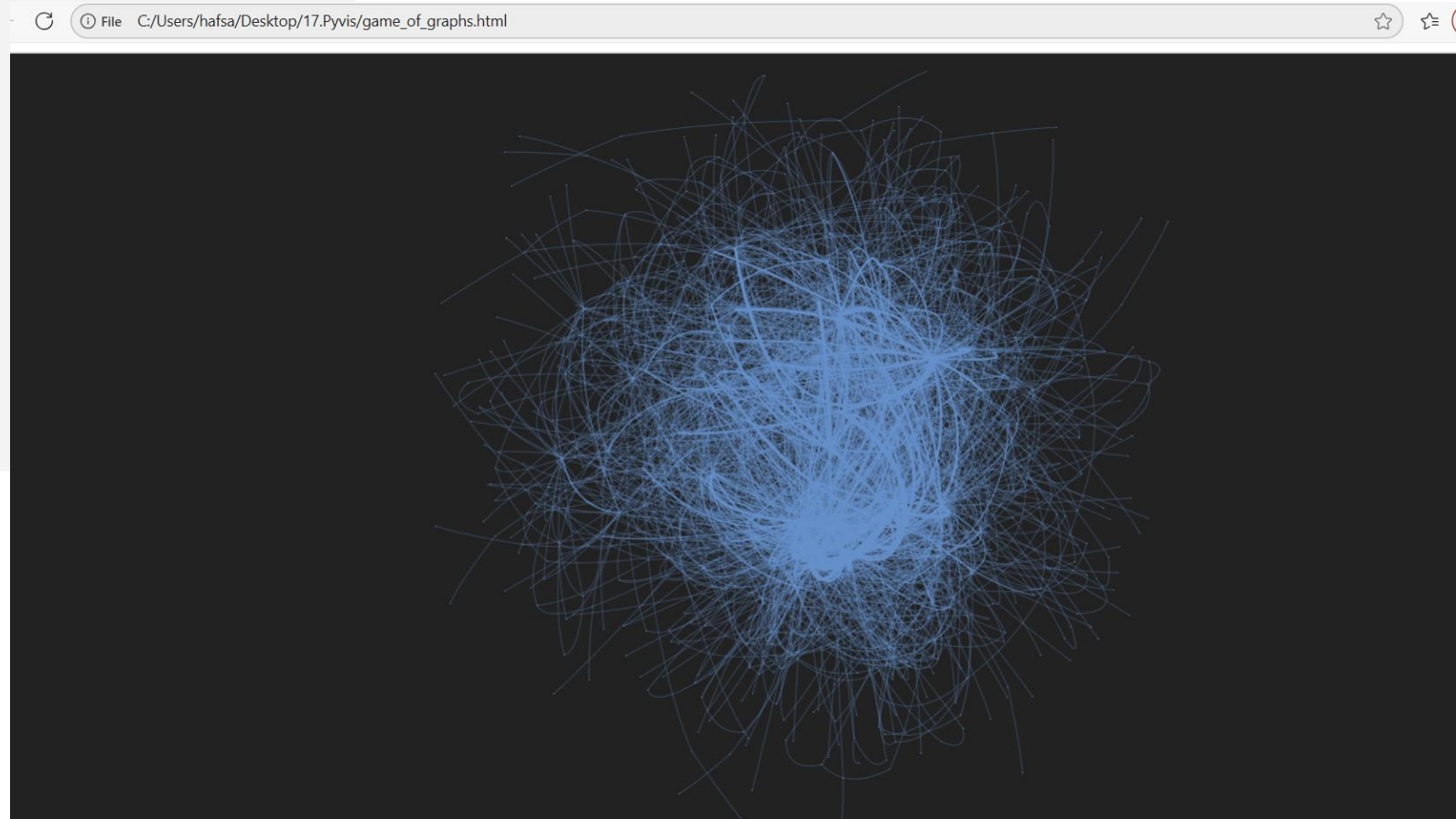




# Game of Graphs visualization

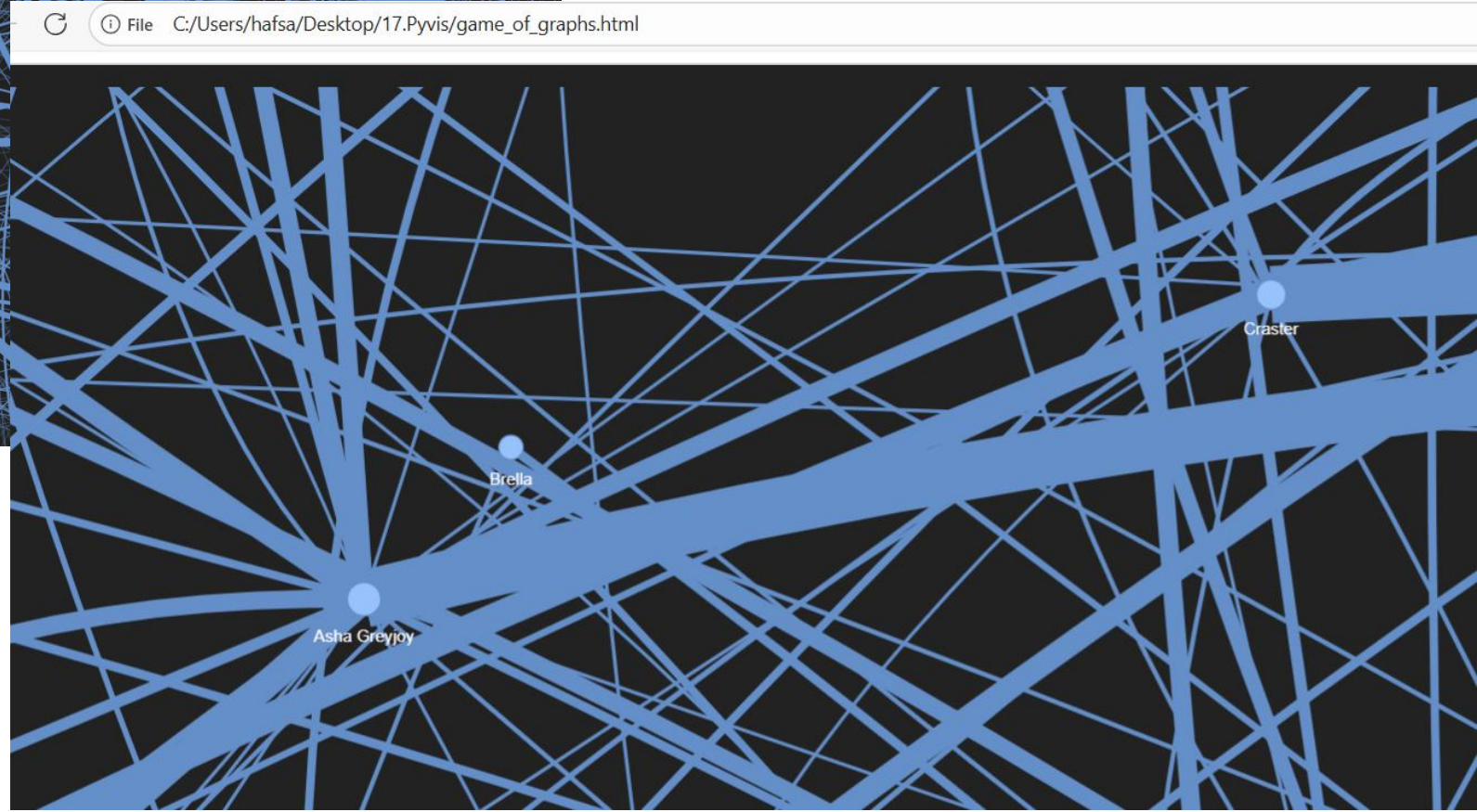
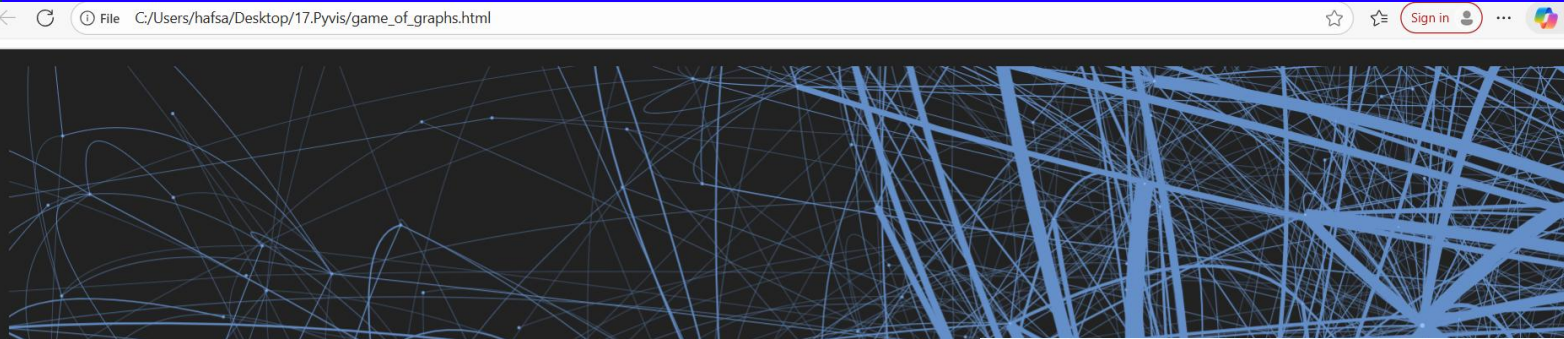
```
from pyvis.network import Network
import pickle
with open("/home/akebli/social_computing/17.Pyvis/game_of_graphs.pkl",
          "rb") as f:
    game_of_graphs = pickle.load(f)
nt = Network(
    height="750px",  # instead of 100%
    width="100%",
    bgcolor="■"#222222",
    font_color="white"
)
nt.barnes_hut()
nt.from_nx(game_of_graphs)
neighbor_map = nt.get_adj_list()
for node in nt.nodes:
    node["value"] = len(neighbor_map[node["id"]])
nt.write_html("game_of_graphs.html")
```

✓ 0.2s





# Game of Graphs visualization





# Why Graph Measures Matter

- Visualizing a network helps us see structure
- But we also need *quantitative measures* to **analyze node importance** and **network behavior**
- Graph measures describe how nodes are connected, central...
- They help us to know:
  - ***Who is the most connected node***
  - **The nodes linking different groups**
  - **The most important nodes in the whole network**

# Function for Graph Measure Visualization

- To compare graph measures visually, we define a **custom draw() function**.
- It **colors the nodes** based on the selected **graph measure** (e.g., Degree, Betweenness, PageRank).
- The **color intensity** shows the **value of the measure** for each node.
- This function can be reused for **any metric**, adapting automatically to the measure provided.

```
import matplotlib.pyplot as plt
✓ def draw(G, pos, measures, measure_name):
✓     nodes = nx.draw_networkx_nodes(G, pos, node_size=150, cmap=plt.cm.plasma,
                                     node_color=list(measures.values()),
                                     nodelist=measures.keys())
     edges = nx.draw_networkx_edges(G, pos)
     plt.title(measure_name)
     plt.colorbar(nodes)
     plt.axis('off')
     plt.show()
```

```
print(nx.degree_centrality(game_of_graphs))
```

✓ 0.0s

```
{'Addam-Marbrand': 0.01509433962264151, 'Aegon-Frey-(son-of-Stevron)': 0.005031446540880503, 'Aegon-I-Targaryen': 0.011320754716981131,
```

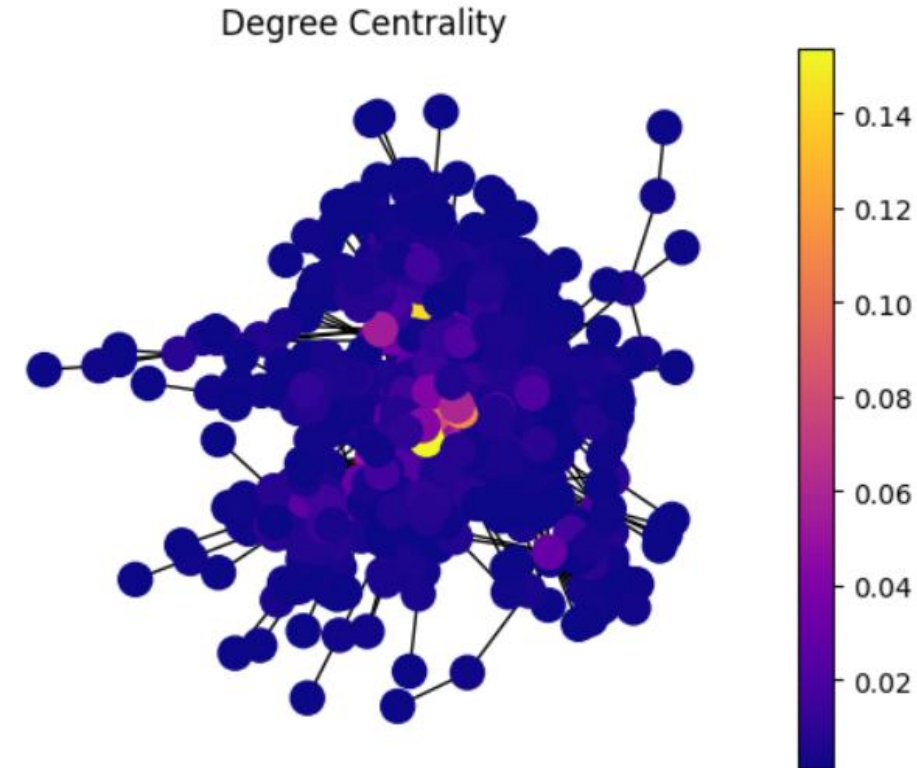
- **Degree Centrality** measures how many **direct connections** each node has.
- It tells us who is **most connected** in the network. The value is the **fraction of nodes** a node is connected to (between 0 and 1).
- In social networks it means “Who interacts with the most people?”
- The brighter nodes represent characters with more connections

```
print(nx.degree_centrality(game_of_graphs))
```

✓ 0.0s

```
{'Addam-Marbrand': 0.01509433962264151, 'Aegon-Frey-(son-of-Stevron)': 0.005031446540880503, 'Aegon-I-Targary
```

```
draw(
    game_of_graphs,
    nx.spring_layout(game_of_graphs),
    nx.degree_centrality(game_of_graphs),
    'Degree Centrality'
)
```



## Betweenness Centrality

- Measures how often a node lies **on the shortest paths** between other nodes.
- Finds **bridge nodes** that connect different parts of the network.
- A **high value** means the node is an **important connector**.
- Brighter nodes** = more central in linking groups.

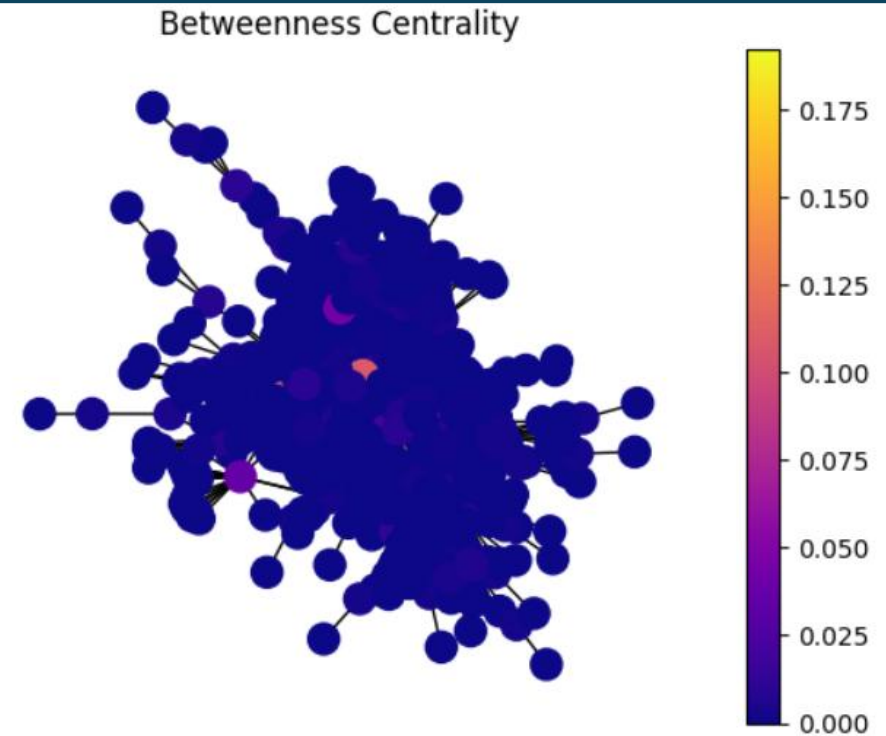
```
print(nx.betweenness centrality(game_of_graphs))
```

23] ✓ 1.3s

```
.. {'Addam-Marbrand': 0.0004696816928106862, 'Aegon-Frey-(son-of-Stevron)': 0.0,
```

```
> ~
draw(
    game_of_graphs,
    nx.spring_layout(game_of_graphs),
    nx.betweenness centrality(game_of_graphs),
    'Betweenness Centrality'
)
```

24] ✓ 4.0s



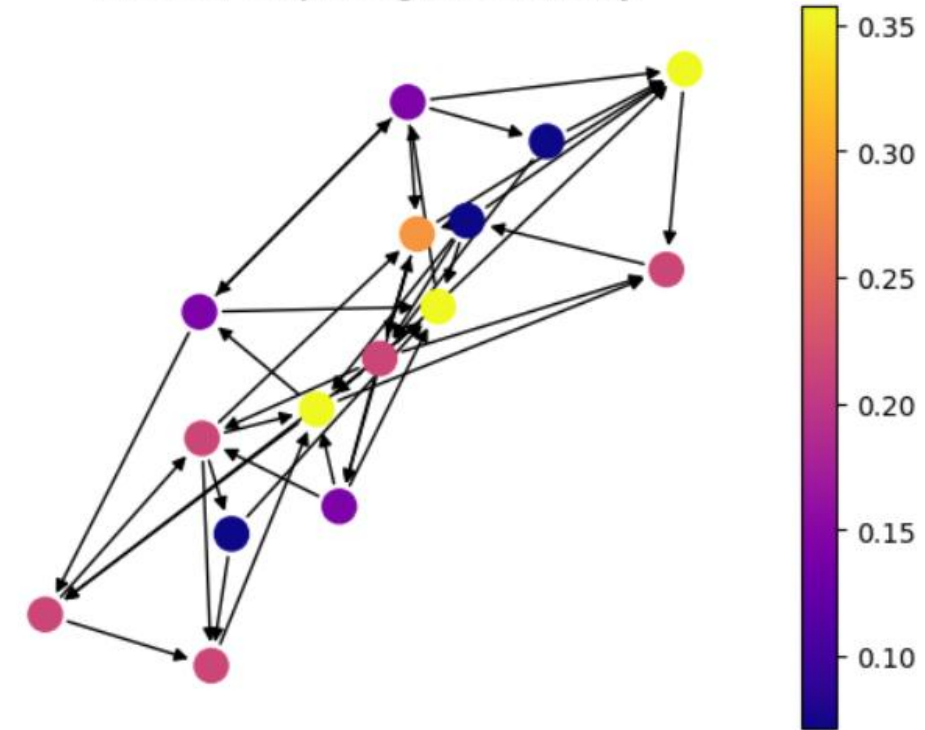
## In-Degree Centrality

- Used for **directed graphs** (where edges have a direction).
- **In-degree centrality** counts how many **incoming connections** a node has.
- It shows **which nodes receive the most links or attention**.
- **Brighter nodes** = more incoming edges (more popular or influential).

```
graph = nx.fast_gnp_random_graph(15, 0.25, directed=True)
draw(
    graph,
    nx.spring_layout(graph),
    nx.in_degree_centrality(graph),
    'Directed Graph Degree Centrality'
)
```

✓ 0.2s

Directed Graph Degree Centrality



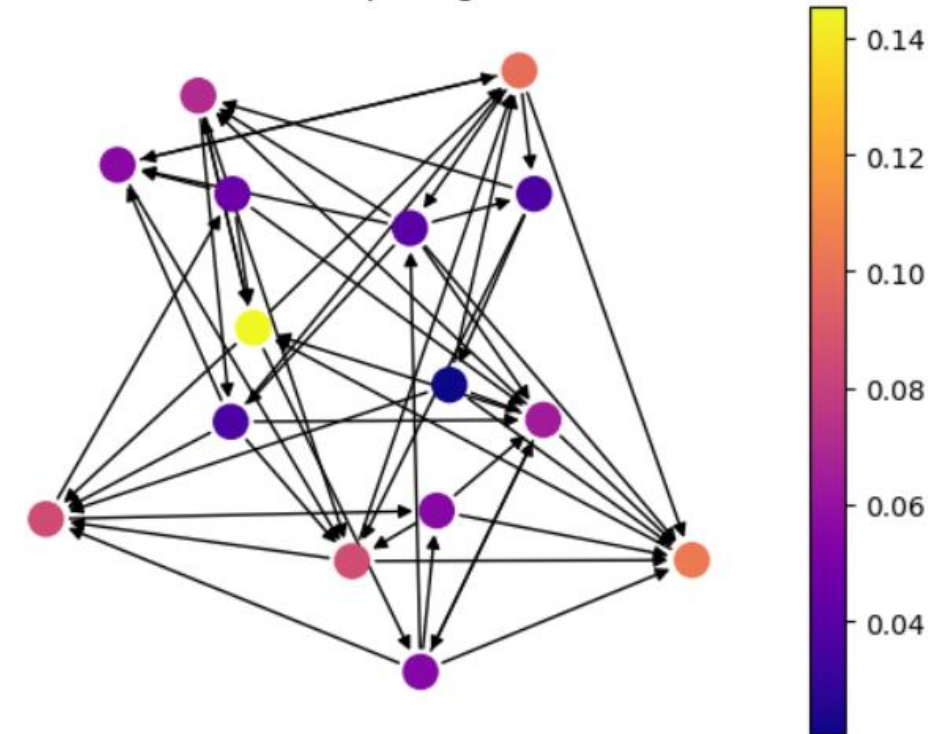
# Graph Measures

- **PageRank** was first developed by **Google** to rank **web pages**.
- **PageRank** estimates **how often a node would be reached** if someone randomly moves through the network.
- A node is **important** if it receives many links, especially from other **highly connected nodes**.
- **Brighter nodes** = nodes that are visited most often in the network.

```
graph = nx.fast_gnp_random_graph(15, 0.25, directed=True)
draw(
    graph,
    nx.spring_layout(graph),
    nx.pagerank(graph, alpha=0.85),
    'Directed Graph PageRank'
)
```

✓ 0.2s

Directed Graph PageRank



# Graph Measures

- A **clique** is a group of nodes where **everyone is connected to everyone else**.
- The **maximum clique** is the **largest** such fully connected group in the network.
- It often represents a **core community** or **strongly linked group** (e.g., main characters or collaborators).
- Here we use `nx.max_clique()` to extract and visualize that subgraph.

```
from networkx.algorithms.approximation import clique
max_cl = clique.max_clique(game_of_graphs)
clique_subgraph = game_of_graphs.subgraph(max_cl)
layout = nx.spring_layout(game_of_graphs)
nx.draw_networkx(clique_subgraph, pos=layout)
plt.show()
plt.close()
```

✓ 1m 28.3s

